

Software Engineering best practices for research software

Dr. Inga Ulusoy, Scientific Software Center
June 2026

Research software

“... software that is developed and used in the context of research...”

Shifting requirements

A scientific question is answered using computation/simulation, but the way the problem is solved changes as part of the research process.

Passed along researchers

Initially developed for one purpose but then often organically extended depending on the researcher's needs.

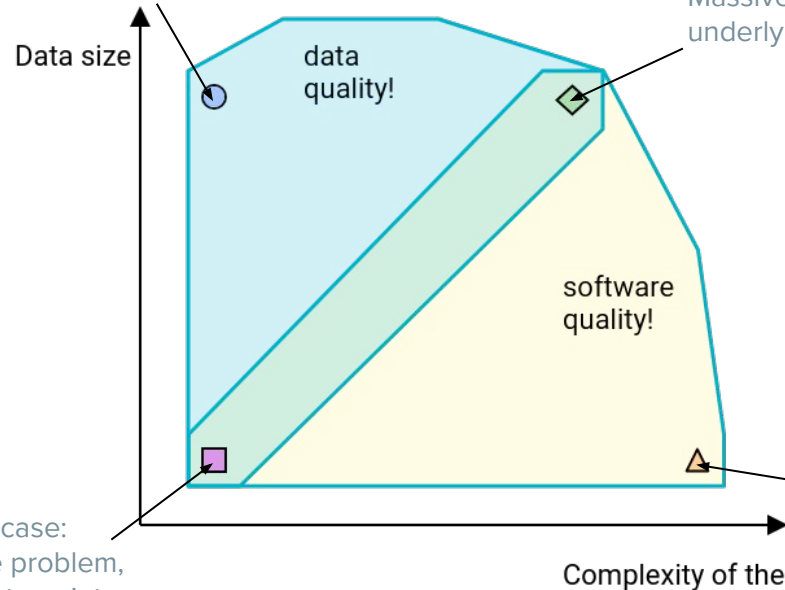
Development Practices

Often created by researchers that have no fundamental training in software engineering and inherit practices from those around them.

Research software and research data

Mathematically simpler problem,
but large amount of data

Massive data and complex
underlying mathematical structure



Machine-learning based RS:
Data **and** software quality
can provide limits to quality
and impact of research!

Data

Data is foundation for..

*...model training, decision making
and/or predictions.*

Different kinds of data

*For example, numerical data, textual
data, images, audio, video.*

Metadata

*What is relevant metadata and should be
included on the data card?*

Availability and licensing

*Will the data be publicly available to the
community? What license does/will the
dataset have?*

Legal considerations

*Where does the data come from? Is it licensed? Is
it public or private data? In what form is the data
stored and processed?*

Ethical considerations

*Does the data exploit work of others? Does
it break some sort of confidentiality? Will it
impact in a possible harmful way or can it be
misconstrued to do harm?*

Bias

*Is there an inherent bias in the data itself,
due to the data collection approach, or other
reasons?*

Software

Purpose

Will the software be more widely used, be an in-house code, or one-person software?

Software engineering best practices

Does the software follow software engineering best practices (version control, testing, documentation, ...)?

Usability and reproducibility

Does the software include documentation on how models can be trained, and keeps track of training parameters? Does the software help to generate model cards and provide models in transferable format?

Accuracy and reliability

Does the software create robust and consistent results, even though it is based on a non-deterministic process?

Legal considerations

Does the software incorporate third-party models and/or code?

Legal considerations

*What license is the software published under?
What license are models published under?*

Security

Is the software secure against data injection?

Best practices short version

Version Control: git

What is this?

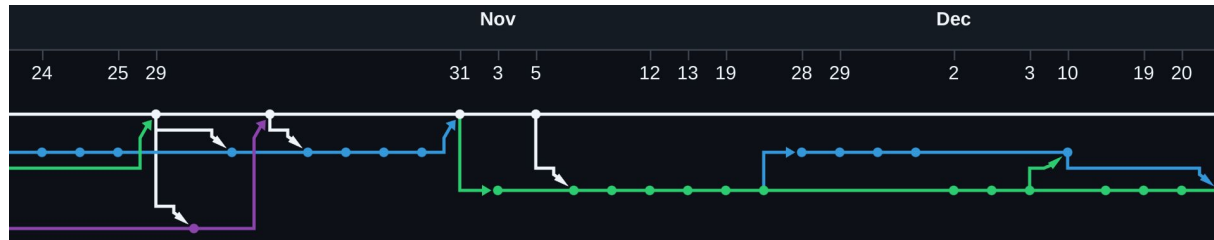
A tool to allow you to track and revert changes, and collaborate with others (change management).

Why is this important?

- Allow versioning of the code and continuing functionality.
- Allow simultaneous changes to the same files.
- Fundamental for reproducing historic states in the line of development.
- Development follows a story and allows other users to build confidence in your work.

Version Control in practice: git

- Create a repo on GitHub
- Clone the repo to your local machine
- Checkout a branch and make changes
- `git add`, `git commit` and `git push`: IDEs such as VSCode make it easy for you!
- Observe how your repo changes on GitHub
- GitHub offers great learning labs: GitHub skills <https://skills.github.com/>
- roadmap.sh offers a git roadmap: <https://roadmap.sh/git-github>



Development workflows: GitHub-flow

What is this?

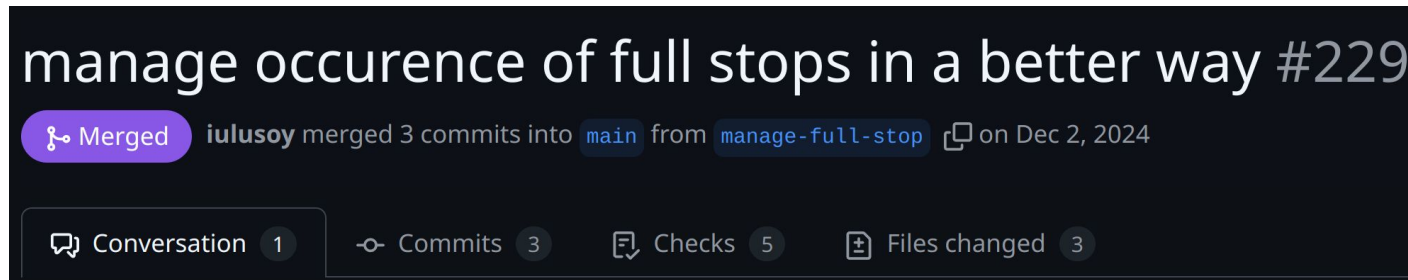
GitHub-flow is a lightweight workflow with creating branches, making changes, opening Pull Requests, running Continuous Integration, and requesting code review, together with Issues and Kanban project boards.

Why is this important?

- Manage how changes are incorporated in the software.
- Track progress in your project and highlight bottlenecks.
- Adhere to development guidelines, ensuring the implementation follows defined rules to ensure software quality.

Development workflows in practice: GitHub-flow

- After you made changes in your git branch, open a Pull Request on GitHub
- Observe how the PR highlights the changes in the line of development
- You can link issues, comment on the PR and run automated checks
- See GitHub skills <https://skills.github.com/>
- See roadmap.sh <https://roadmap.sh/git-github>



Quality management: Testing and continuous Integration



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

What is this?

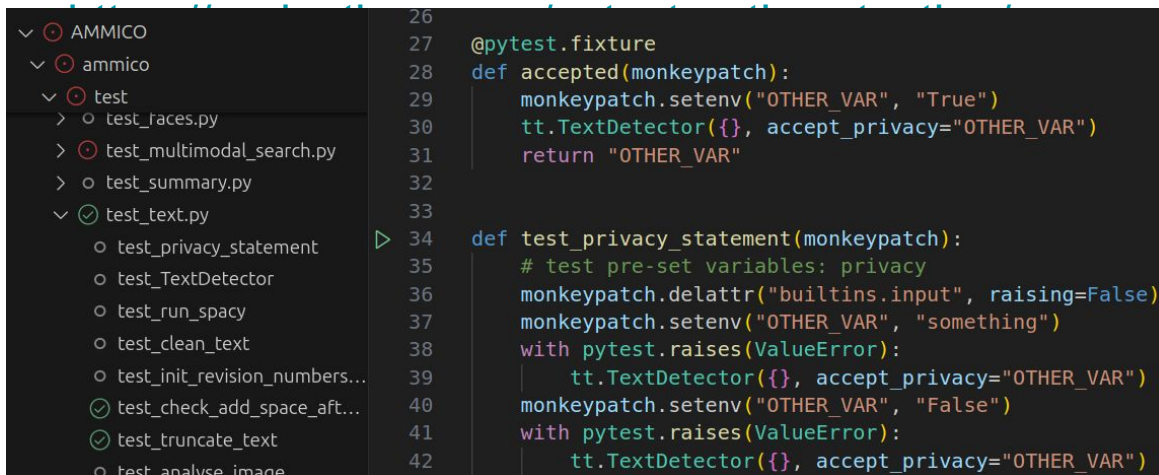
Quality management ensures that the software is producing correct results and that the results are reproducible.

Why is this important?

- As a scientist, you should take utmost care to conduct your science in accordance with the scientific code of conduct.
- Yourself or other researchers may need to reproduce results for further studies.
- Ensure that nothing breaks when you implement changes/new features.

Testing in practice

- Use testing frameworks such as pytest
- Write tests in a tests/ folder: unit tests, integration tests, system tests, compatibility tests, ...
- To learn how to use pytest: <https://docs.pytest.org/en/stable/>,



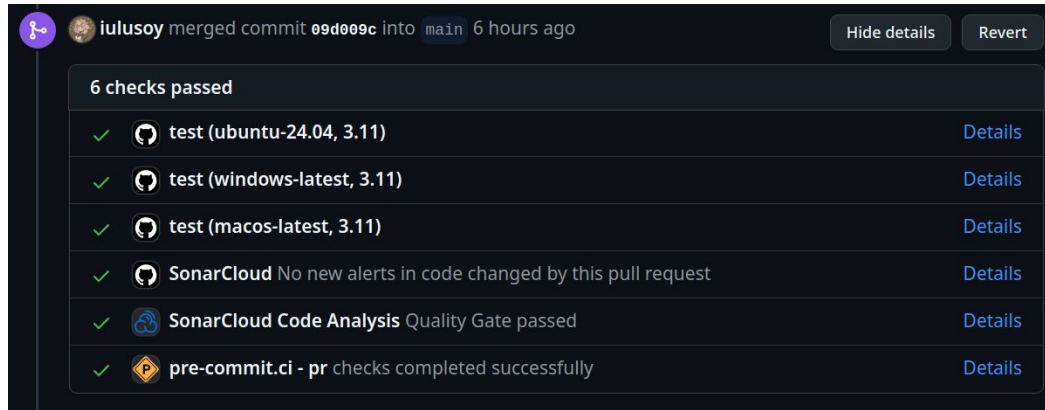
```

26
27 @pytest.fixture
28 def accepted(monkeypatch):
29     monkeypatch.setenv("OTHER_VAR", "True")
30     tt.TextDetector({}, accept_privacy="OTHER_VAR")
31     return "OTHER_VAR"
32
33
34 def test_privacy_statement(monkeypatch):
35     # test pre-set variables: privacy
36     monkeypatch.delattr("builtins.input", raising=False)
37     monkeypatch.setenv("OTHER_VAR", "something")
38     with pytest.raises(ValueError):
39         tt.TextDetector({}, accept_privacy="OTHER_VAR")
40     monkeypatch.setenv("OTHER_VAR", "False")
41     with pytest.raises(ValueError):
42         tt.TextDetector({}, accept_privacy="OTHER_VAR")

```

Continuous integration in practice

- Set up your tests to be automatically run by GitHub actions
- Include code linter and quality control in your actions
- These should be set up to run automatically when you open a Pull Request
- GH actions, codecov, sonarcloud, snyk, pre-commit, code formatting (black), GitHub Guardian, dependabot



Documentation

What is this?

Documentation can be comments, docstrings, readme's, tutorials, demonstration notebooks, and contains technical and domain-specific / application-specific descriptions of the software.

Why is this important?

- Document what the software can and cannot do, and parameter ranges.
- Allow others to install and use your software (or yourself, at a later time).
- Allow others to contribute to your software.

Documentation in practice



text module

```
class text.PostprocessText(mydict: dict | None = None, use_csv: bool = False, csv_path: str | None = None,
analyze_text: str = 'text_english')
```

Bases: `object`

```
analyse_topic(return_topics: int = 3)→ tuple
```

Performs topic analysis using BERTopic.

Parameters: `return_topics` (*int, optional*) – Number of topics to return. Defaults to 3.

Returns: `tuple` – A tuple containing the topic model, topic dataframe, and most frequent topics.

```
get_text_df(analyze_text: str)→ list
```

Extracts text from the provided dataframe.

Parameters: `analyze_text` (*str*) – Column name for the text field to analyze.

Returns: `list` – A list of text extracted from the dataframe.

```
get_text_dict(analyze_text: str)→ list
```

Extracts text from the provided dictionary.

Parameters: `analyze_text` (*str*) – Key for the text field to analyze.

Returns: `list` – A list of text extracted from the dictionary.

- Use tools like sphinx and mkdocs to render docstrings and markdown at minimal effort
- Include jupyter notebooks that showcase use of your software - these can be run on google colab
- Document dependencies in a requirements file and provide installation instructions

Deployment: Runtime environment / containerisation



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

What is this?

Deployment information such as runtime environments or containers allow easy adaption as they provide direct access to running the software without installation and dependency conflicts.

Why is this important?

- A big step towards reproducibility and transferability of your approach.
- The software ecosystem changes quickly, and this allows to preserve a snapshot that can be shared and run easily.

Containerisation in practice



- Use docker to provide build instructions for containers, and possibly deploy the containers on Dockerhub for anyone to download and use
- Docker roadmap <https://roadmap.sh/docker>, official tutorial <https://docs.docker.com/>

```
Dockerfile > FROM
1 FROM jupyter/base-notebook
2
3 # Install system dependencies for computer vision packages
4 USER root
5 RUN apt update && apt install -y build-essential libgl1 libglb2.0-0 libsm6 libxrender1 libxext6
6 USER $NB_USER
7
8 # Copy the repository into the container
9 COPY --chown=${NB_UID} . /opt/ammico
10
11 # Install the Python package
12 RUN python -m pip install /opt/ammico
13
14 # Make JupyterLab the default for this application
15 ENV JUPYTER_ENABLE_LAB=yes
16
17 # Export where the data is located
18 ENV XDG_DATA_HOME=/opt/ammico/data
```

Software Licensing

What is this?

A software license states the terms of use, re-use and distribution, among others, without violating copyrights, and defines responsibilities.

Why is this important?

- So that others may use your code, and to prevent misuse.
- So that others may contribute to your code.
- So that the responsibilities for how the software is used are clear.
- Establishes the rights of all parties involved with the software.

Software licensing in practice

- Use the provided templates from GitHub: Either at repository creation or when adding a new file called LICENSE
- Permissive open-source license: BSD 2-Clause, MIT, Apache License 2.0
- Copyleft open-source license: GNU version 3, LGPL
- Proprietary licenses: Do not only keep your project close-source and potentially less visible, but also carry responsibilities for contract fulfillment
- <https://opensource.org/licenses>, <https://choosealicense.com/>
- **When using/incorporating third-party software:** ie. use of open-source libraries - is the third-party code distributed with your software? If so, compatibility needs to be confirmed!

Starting a new project

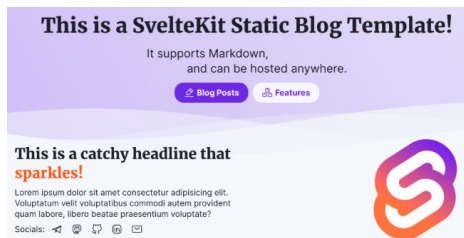
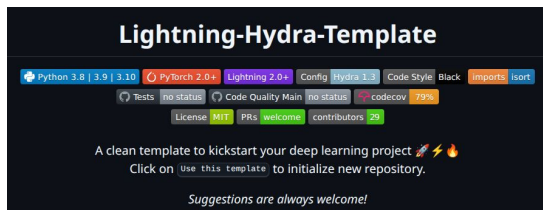
Project templates

What is this?

Project templates exist for example on GitHub. They include configurations for specific types of projects.

Why is this important?

- Project templates help you save time by providing complete configurations for testing, CI, and documentation, for example.
- Templates help you adhere to common practices, configurations, or standards.



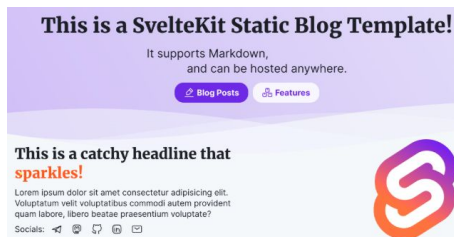
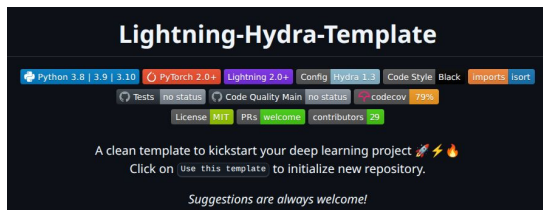
Cookiecutters

What is this?

A cookiecutter is a reusable project template that automatically generates a new codebase with a predefined structure, configuration, and files.

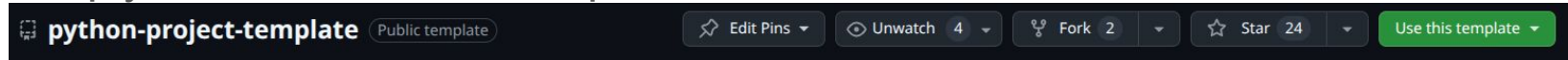
Why is this important?

- Project templates help you save time by providing complete configurations for testing, CI, and documentation, for example.
- Templates help you adhere to common practices, configurations, or standards.



Project templates in practice

- For example: GitHub template repositories
- Simply click on “use this template”



- The SSC offers such template repositories for
 - C++
 - Python
 - Fortran
 - Matlab
- Consult our guidelines <https://ssciwr.github.io/guidelines/>



Cookiecutters in practice

- For example: BioCypher cookiecutter

```
# Install cookiecutter
pip install cookiecutter

# Generate project from template
cookiecutter https://github.com/biocypher/biocypher-cookiecutter-template.git
```

- Fill in the metadata as prompted or run using the BioCypher MCP

Generated Project Structure

```
my-biocypher-pipeline/
├── config/
│   ├── biocypher_config.yaml
│   └── schema_config.yaml
├── src/my_biocypher_pipeline/
│   ├── __init__.py
│   └── adapters/
│       ├── __init__.py
│       └── my_resource_adapter.py
├── tests/
│   ├── __init__.py
│   └── test_my_resource_adapter.py
├── create_knowledge_graph.py
├── docker-compose.yml
├── Dockerfile
├── pyproject.toml
├── README.md
└── .gitignore
```


Publishing a project

Software publication

What is this?

A software publication associates a DOI or other persistent identifier with your software.

Why is this important?

- This makes your software citable in scientific publications.
- You can reference your software plus providing the version number that you used.
- Your contribution to the scientific community becomes more visible.
- The publication helps you to structure the code and documentation for a user's perspective.

Software publication in practice

- A list of software-specific journals:
<https://www.software.ac.uk/top-tip/which-journals-should-i-publish-my-software>
- Write a short paper detailing your software, the architecture and design, in- and output, and purpose of the software
- Another option is to place a snapshot of your repo on zenodo, this also allows you to obtain a DOI
- Another option is to upload your project to Software Heritage and obtain a persistent SWHID <https://docs.softwareheritage.org/devel/swh-model/persistent-identifiers.html>

We are building the universal software archive

Software Heritage



Collect
Preserve
Share

We **collect** and **preserve** software in source code form, because software embodies our technical and scientific knowledge and humanity cannot afford the risk of losing it.

Software is a precious part of our cultural heritage. We curate and make accessible all the software we collect, because only by **sharing** it we can guarantee its preservation in the very long term.

[Browse the archive](#)

[Discover our mission](#)



The Journal of
Open Source Software



WIREs COMPUTATIONAL MOLECULAR SCIENCE

Data publication

What is this?

Data can be published in dedicated data repositories and obtain a DOI.

Why is this important?

- You can publish your datasets, making them available to the community.
- By associating a DOI, you may garner citations.
- Data often is the foundation for reproducing results.
- Data should be stored and preserved separately from the software once it reached a stage that it does not change anymore.

Data publication in practice

- Option A: Publish your data on zenodo and obtain a DOI
- Option B: Publish your data in your institutionally provided data repository, ie heiDATA and obtain a DOI
- Option C: Publish your data on community platforms such Hugging Face *if these provide a DOI*
- Make sure your data is clean and does not need to be updated anymore
- Cite your own data repository in your publications
- You can also download your own data dynamically within your code, ie,. using pooch <https://www.fatiando.org/pooch/dev/> or datasets <https://huggingface.co/docs/datasets/en/index>





Contributing to an Open-Source Software

Contributing to Open-Source Software

What is this?

Open-source software critically depends on contributions from the community.

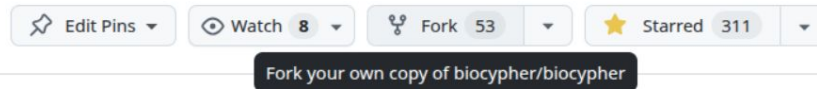
Why is this important?

- Usually just one or a few developers responsible for a software - they cannot test all possible use cases. You can help improve the software by flagging and resolving issues.
- The dev team usually also has other responsibilities and cannot maintain the software only by themselves - give back to the community.
- You can improve your technical skills and build a portfolio for your own profile.

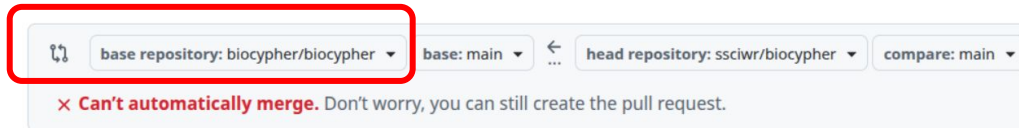
Contributing to Open-Source Software in practice

- Start by identifying how you can contribute:
 - Look at open issues, there are specific labels such as “good first issue”
 - Or if you have noticed an issue by yourself, you can open one and start resolving it
 - Consult the developer guidelines to check what you should consider for your contribution, and how the contribution process is structured

- Create a fork of the project



- Open a branch in your fork and make changes in that branch
- Open a PR **to your own main branch** in your fork and confirm that all checks pass



Do not open a PR to the upstream! Select your fork here

Contributing to Open-Source Software in practice



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

- If you are sure about your contribution, merge the PR with your main branch
- Then open a PR from your main branch to the upstream repository
- Make sure to include all information that is needed, such as a good description, and check the guidelines again
- You can tag a developer for review or request review. By checking the little box “allow edits by maintainers” the repo maintainers can also make changes to your PR
- Allow some time for the maintainers to review your PR, and work together with them to get your changes merged
- That’s it!

Notifications

[Customize](#)

Unsubscribe

You're receiving notifications because you authored the thread.

1 participant



☒ Allow edits by maintainers

Summary

Research Software: A special type of software

Research Data: Interconnected to your software

Research publication: Make your software and data available to others

Reproducibility: Strictly only possible if software and data is available

Summary

Best practices: Building blocks for your software project

